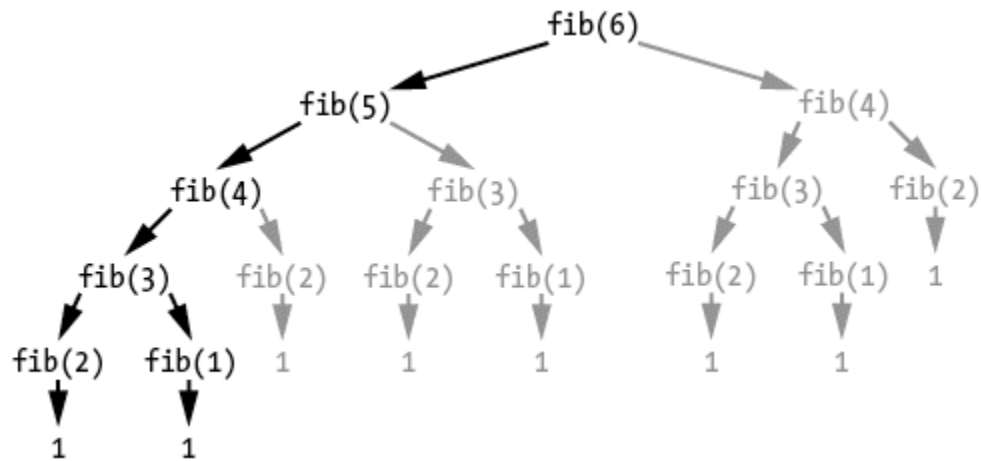




*Figure 2-3: A tree diagram of the recursive function calls made starting with `fibonacci(6)`. The redundant function calls are in gray.*

This causes a cascade of other function calls until they reach the base cases of `fibonacci(2)` and `fibonacci(1)`, which return 1. But notice that `fibonacci(4)` is called twice, and `fibonacci(3)` is called three times, and so on. This slows the overall algorithm with unnecessarily repeated calculations. This inefficiency gets worse as the Fibonacci number you want to calculate gets larger. While the iterative Fibonacci algorithm can complete `fibonacci(100)` in less than a second, the recursive algorithm would take over a million years to complete.